

Technical Note: block-VIEM.jl

Volume Integral Equation Method with SWG/half-SWG Basis,
Duffy Singular Integration, and AIM-FFT Acceleration
for Light Scattering by Arbitrary Dielectric Particles

N. Moteki

Last updated: 2026-04-24 (corresponds to block-VIEM.jl v0.7.7)

Contents

1	Introduction	3
2	VIEM Formulation	4
2.1	EFVIE-D strong form	4
2.2	Incident plane wave	4
2.3	SWG and half-SWG basis functions	5
2.4	Galerkin weak form	5
3	Weakening the Singularity via Integration by Parts	6
3.1	Bulk–bulk term K^A	6
3.2	Surface correction terms K^B, K^C, K^D	6
3.3	Charge neutrality	7
3.4	Mass matrix	7
4	Singular and Near-Singular Volume Integration	7
4.1	Generalised vertex Duffy transform	7
4.2	Pair classification	8
4.3	Surface integrals for K^B, K^C, K^D	8
5	FFT-Accelerated MVP: Adaptive Integral Method	8
5.1	Auxiliary grid and stencil	8
5.2	Moment matching	8
5.3	Far-field product via FFT	9
5.4	Precorrection of near-field pairs	9
5.5	Surface-moment extension for K^B, K^C, K^D	9
6	Solvers	12
6.1	Direct and Krylov solvers	12
6.2	Multi-orientation batch solve	12
6.3	Parallel execution	13
7	Scattering Observables	13
7.1	Far-field scattering amplitude	13
7.2	Absorption cross section	14
7.3	Scattering cross section by far-field integration	14
7.4	PCAS forward amplitude	14
7.5	OCBS backward amplitude	14

8	Validation	15
8.1	Oblate spheroid α -symmetry	15
8.2	Accuracy and computational cost vs. block-DDA_Py	16
8.3	Two-sphere doublet vs MSTM (multi-orientation)	17
9	Spheroid Sweep HDF5 Output	18
9.1	Parameter grid	18
9.2	Per-wavelength datasets	18
10	AIM-accelerated block-Krylov multi-orientation solve	19
11	Particle shape models and cross-validation	20
12	Anisotropic Permittivity	20
12.1	Modified weak form	20
12.2	Implementation	21
12.3	Cross-validation vs block-DDA_Py	21

1 Introduction

`block-VIEM.jl` is a Julia implementation of the volume integral equation method (VIEM) for electromagnetic scattering by arbitrarily shaped, high-contrast dielectric particles. It is designed as a higher-accuracy successor to the discrete dipole approximation (DDA) code `block-DDA_Py` [11], targeting scattering problems where DDA does not converge or provides insufficient accuracy — for example, particles with high refractive-index contrast (e.g. plasmonic metals), extreme aspect ratios, or surface roughness features smaller than the DDA lattice spacing.

The key features of `block-VIEM.jl` are:

- **Higher accuracy per degree of freedom than DDA.** Tetrahedral mesh discretization with SWG (half-SWG) basis functions [1] and Duffy-transform singular integration achieves 10–12× better accuracy per DOF than cubic-lattice DDA on equivalent problems.
- **AIM/FFT-accelerated block-Krylov solver.** The Adaptive Integral Method (AIM) [5] computes matrix–vector products in $O(N \log N)$ via FFT. Block BiCGSTAB and Block GMRES solve for all particle orientations simultaneously.
- **Multi-orientation batch solve with spheroid mode.** Deterministic Euler angle grids on $SO(3)$ identical to `block-DDA_Py`. For spheroids, only N_β orientations are solved; the full grid is filled analytically.
- **Birefringent (anisotropic) materials.** Diagonal permittivity tensor $\epsilon_p = \text{diag}(\epsilon_x, \epsilon_y, \epsilon_z)$.
- **Two particle-shape families with automatic mesh generation.** Gaussian Random Ellipsoid (GRE, parametric irregular shapes matching `block-DDA_Py`) and sphere aggregates (chains, planar arrays, FCC/BCC/HCP clusters, PTSA fractal-like aggregates). Both are meshed with adaptive tetrahedral resolution via Gmsh and a volume-preserving final rescale (specified in `docs/descriptions_particle_shape_model.md`).
- **CAS-v2 observables and block-DDA_Py-compatible I/O.** Forward/backward complex scattering amplitudes (PCAS, OCBS), cross sections, and HDF5 parameter-sweep output in the same schema as `block-DDA_Py`.

The two codes share the same physical input conventions, HDF5 output schema, and CAS-v2 observable definitions, so downstream analysis pipelines work with either without modification.

Scope of this note. This document describes the theoretical background and algorithms implemented in `block-VIEM.jl`, covering:

1. the EFVIE-D formulation with SWG and half-SWG basis functions (Sec. 2),
2. weakening the $1/R^3$ singularity to $1/R$ via integration by parts, including the boundary-face surface-correction terms K^B , K^C , K^D required to restore charge neutrality (Sec. 3),
3. evaluation of the remaining $1/R$ singular and near-singular volume integrals via the generalized Duffy transform (Sec. 4),
4. FFT-accelerated matrix–vector products through the Adaptive Integral Method (AIM) with moment matching and precorrection (Sec. 5),
5. the direct and Krylov iterative solvers and the multi-orientation batch solve (Sec. 6),
6. the CAS-v2 compatible scattering observables computed from the solution (Sec. 7),
7. validation against Mie theory and the oblate-spheroid symmetry identity (Sec. 8),
8. the HDF5 output format for spheroid parameter sweeps, bit compatible with the `block-DDA_Py` schema consumed by `build_spheroid_lut.py` (Sec. 9).

Notation and conventions. Throughout this note the electromagnetic equations are written in the physics (BH83 [10]) time convention $e^{-i\omega t}$, so that an outgoing scalar Helmholtz Green's function carries the factor $e^{+ik_0 R}/(4\pi R)$. Absorbing materials have $\text{Im}(\epsilon_p) > 0$. Lengths are expressed in micrometres (μm). Vectors are set in bold italic ($\mathbf{E}, \mathbf{k}, \mathbf{r}$), matrices and second-rank tensors in upright bold ($\mathbf{A}, \mathbf{G}$), and unit vectors carry a hat ($\hat{\mathbf{e}}, \hat{\mathbf{n}}$).

2 VIEM Formulation

2.1 EFVIE-D strong form

Consider a non-magnetic ($\mu = \mu_0$) inhomogeneous dielectric particle occupying the volume Ω with complex permittivity $\epsilon_p(\mathbf{r})$, embedded in a homogeneous background of real permittivity ϵ_{bg} . The wavenumber in the background is

$$k_0 = \omega \sqrt{\mu_0 \epsilon_{\text{bg}}} = \frac{2\pi m_m}{\lambda_0}, \quad (1)$$

where $m_m = \sqrt{\epsilon_{\text{bg}}/\epsilon_0}$ is the refractive index of the background medium.

We use the electric flux volume integral equation in the *D-form* [1]: the unknown field is the electric flux density $\mathbf{D}(\mathbf{r}) = \epsilon_p(\mathbf{r}) \mathbf{E}(\mathbf{r})$, whose normal component is continuous across material interfaces, so the scheme places all dielectric jumps inside the basis functions and not in the expansion coefficients. Define the dielectric contrast

$$\kappa(\mathbf{r}) = \frac{\epsilon_p(\mathbf{r}) - \epsilon_{\text{bg}}}{\epsilon_p(\mathbf{r})}. \quad (2)$$

The equivalent volume polarisation current is $\mathbf{J}_v = -i\omega \kappa \mathbf{D}$, and the EFVIE-D strong form reads [1, 2]

$$\mathbf{E}_{\text{inc}}(\mathbf{r}) = \frac{\mathbf{D}(\mathbf{r})}{\epsilon_p(\mathbf{r})} - \frac{1}{\epsilon_{\text{bg}}} (k_0^2 + \nabla \nabla \cdot) \int_{\Omega} \kappa(\mathbf{r}') \mathbf{D}(\mathbf{r}') G(\mathbf{r}, \mathbf{r}') d^3 r', \quad (3)$$

where $G(\mathbf{r}, \mathbf{r}')$ is the free-space scalar Helmholtz Green's function

$$G(\mathbf{r}, \mathbf{r}') = \frac{e^{+ik_0 R}}{4\pi R}, \quad R = |\mathbf{r} - \mathbf{r}'|. \quad (4)$$

2.2 Incident plane wave

The incident field is a monochromatic plane wave

$$\mathbf{E}_{\text{inc}}(\mathbf{r}) = \mathbf{E}_0 \exp(+ik_0 \hat{\mathbf{k}}_{\text{inc}} \cdot \mathbf{r}), \quad (5)$$

with $\hat{\mathbf{k}}_{\text{inc}}$ the unit propagation direction and $\mathbf{E}_0$ a complex polarisation vector perpendicular to $\hat{\mathbf{k}}_{\text{inc}}$. For CAS-v2 observables the incident polarisation is left-handed circular (matching block-DDA-Py [11]):

$$\mathbf{E}_0 = \frac{1}{\sqrt{2}} \hat{\mathbf{e}}_{\theta, \text{inc}} + \frac{i}{\sqrt{2}} \hat{\mathbf{e}}_{\phi, \text{inc}}, \quad (6)$$

where $\hat{\mathbf{e}}_{\theta, \text{inc}}, \hat{\mathbf{e}}_{\phi, \text{inc}}$ are the polar and azimuthal unit vectors of the incident direction in the particle frame. Particle orientation is parameterised by intrinsic ZYZ Euler angles (α, β, γ) , following `scipy.spatial.transform.Rotation` and `block-DDA-Py`. We denote by

$$\mathbf{R}(\alpha, \beta, \gamma) = \mathbf{R}_z(\alpha) \mathbf{R}_y(\beta) \mathbf{R}_z(\gamma) \quad (7)$$

the active rotation that maps particle-frame coordinates of a physical vector to its lab-frame coordinates; equivalently, (7) is the fixed-frame composition obtained by applying the three extrinsic rotations $\mathbf{R}_z(\gamma), \mathbf{R}_y(\beta), \mathbf{R}_z(\alpha)$ in that order about the lab axes. The laboratory-frame triad $\{\hat{\mathbf{u}}_{\text{inc}, L}, \hat{\mathbf{e}}_{\theta, L}, \hat{\mathbf{e}}_{\phi, L}\}$ is therefore expressed in the particle frame by applying $\mathbf{R}(\alpha, \beta, \gamma)^\top (= \mathbf{R}(\alpha, \beta, \gamma)^{-1})$ since $\mathbf{R}$ is orthogonal).

2.3 SWG and half-SWG basis functions

The particle volume Ω is discretised into a conforming tetrahedral mesh $\mathcal{T}_h$ with N_{tet} elements. The mesh is generated by Gmsh with a target characteristic edge length ℓ_c , which controls the maximum edge length of every tetrahedron; smaller ℓ_c yields a finer mesh with more elements and degrees of freedom. The two particle-shape families supported (Gaussian Random Ellipsoid and sphere aggregates), the adaptive choice of ℓ_c , the `neck_ratio` neck-width convention, and the volume-preserving final rescale are documented separately in `docs/descriptions_particle_shape_model.md`; the present note proceeds from the assumption that $\mathcal{T}_h$ has been generated and is conforming. On each internal face S_n shared by two tetrahedra T_n^+ and T_n^- , an SWG basis function [1, Eqs. (10), (12)] is defined by

$$\mathbf{f}_n(\mathbf{r}) = \begin{cases} \frac{a_n}{3V_n^+} (\mathbf{r} - \mathbf{p}_n^+), & \mathbf{r} \in T_n^+, \\ -\frac{a_n}{3V_n^-} (\mathbf{r} - \mathbf{p}_n^-), & \mathbf{r} \in T_n^-, \\ \mathbf{0}, & \text{otherwise,} \end{cases} \quad (8)$$

where a_n is the face area, $V_n^\pm$ the volumes of the two supporting tetrahedra, and $\mathbf{p}_n^\pm$ the free vertex (the vertex of $T_n^\pm$ not on S_n). The normalisation in (8) enforces $\mathbf{f}_n \cdot \hat{\mathbf{n}} = 1$ on S_n (oriented from T_n^+ to T_n^-) and zero on the other five faces of $T_n^+ \cup T_n^-$. The divergence is piecewise constant,

$$\nabla \cdot \mathbf{f}_n(\mathbf{r}) = \begin{cases} +\frac{a_n}{V_n^+}, & \mathbf{r} \in T_n^+, \\ -\frac{a_n}{V_n^-}, & \mathbf{r} \in T_n^-, \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

A naive SWG expansion that uses only internal faces implicitly imposes $\mathbf{D} \cdot \hat{\mathbf{n}} = 0$ on the particle boundary $\partial\Omega$, leaving the surface polarisation charge unrepresented and producing systematic cross-section errors of order 10–30% on Mie tests. To restore charge neutrality, the original [1] formulation adds one half-SWG basis function per boundary face. On a boundary face S_n with a single supporting tetrahedron T_n^+ , the half-SWG function is the first branch of (8) alone:

$$\mathbf{f}_n^{\text{half}}(\mathbf{r}) = \frac{a_n}{3V_n^+} (\mathbf{r} - \mathbf{p}_n^+), \quad \mathbf{r} \in T_n^+, \quad (10)$$

with $\mathbf{f}_n \cdot \hat{\mathbf{n}} = 1$ on S_n and zero on the other three faces of T_n^+ . For a half-SWG function the normal flux does *not* vanish on $\partial\Omega$, so it carries an induced surface polarisation charge $\sigma = \kappa \mathbf{D} \cdot \hat{\mathbf{n}}$ whose integral over S_n exactly cancels the bulk bound charge of the same basis (Sec. 3.3). The electric flux density is expanded as

$$\mathbf{D}(\mathbf{r}) \approx \sum_{n=1}^{N_{\text{dof}}} D_n \mathbf{f}_n(\mathbf{r}), \quad N_{\text{dof}} = N_{\text{int. faces}} + N_{\text{bdy. faces}}. \quad (11)$$

2.4 Galerkin weak form

Testing (3) with a basis function $\mathbf{f}_m$ and substituting (11) yields the $N_{\text{dof}} \times N_{\text{dof}}$ linear system

$$\sum_{n=1}^{N_{\text{dof}}} Z_{mn} D_n = b_m, \quad b_m = \int_{\Omega} \mathbf{f}_m(\mathbf{r}) \cdot \mathbf{E}_{\text{inc}}(\mathbf{r}) d^3r, \quad (12)$$

with the Galerkin impedance matrix element

$$\begin{aligned} Z_{mn} = & \int_{\Omega} \frac{\mathbf{f}_m(\mathbf{r}) \cdot \mathbf{f}_n(\mathbf{r})}{\epsilon_p(\mathbf{r})} d^3r \\ & - \frac{1}{\epsilon_{\text{bg}}} \int_{\Omega} \mathbf{f}_m(\mathbf{r}) \cdot \left[(k_0^2 + \nabla \nabla \cdot) \int_{\Omega} \kappa(\mathbf{r}') \mathbf{f}_n(\mathbf{r}') G(\mathbf{r}, \mathbf{r}') d^3r' \right] d^3r. \end{aligned} \quad (13)$$

For a homogeneous particle, κ and ϵ_p are constants and factor out of the inner integral, giving the compact decomposition

$$Z_{mn} = \frac{1}{\epsilon_p} M_{mn} - \frac{\kappa}{\epsilon_{bg}} K_{mn}, \quad (14)$$

where $M_{mn} = \int \mathbf{f}_m \cdot \mathbf{f}_n d^3r$ is the mass matrix and K_{mn} is the radiation kernel defined below.

3 Weakening the Singularity via Integration by Parts

As written in (13), the radiation kernel $(k_0^2 + \nabla \nabla \cdot)G$ contains a hyper-singular term $1/R^3$ coming from $\nabla \nabla \cdot G$. Moving the two derivatives off G onto $\mathbf{f}_m$ and $\mathbf{f}_n$ by successive integrations by parts reduces the singularity to the weakly singular $1/R$ kernel that Duffy quadrature can handle. Because SWG and half-SWG functions are only piecewise smooth and only half-SWG functions have nonzero $\mathbf{f}_n \cdot \hat{\mathbf{n}}$ on $\partial\Omega$, the by-parts procedure generates three boundary-face contributions in addition to the standard bulk term.

3.1 Bulk–bulk term K^A

Apply $\nabla \nabla \cdot$ to the inner integral and move the inner divergence onto $\mathbf{f}_n$ using the divergence theorem; do the same for the outer divergence and $\mathbf{f}_m$. For internal SWG basis functions (vanishing normal flux on $\partial\Omega$) the boundary terms are identically zero and the resulting bulk kernel is

$$K_{mn}^A = \int_{\Omega} \int_{\Omega} [k_0^2 \mathbf{f}_m(\mathbf{r}) \cdot \mathbf{f}_n(\mathbf{r}') - (\nabla \cdot \mathbf{f}_m)(\mathbf{r}) (\nabla \cdot' \mathbf{f}_n)(\mathbf{r}')] G(\mathbf{r}, \mathbf{r}') d^3r' d^3r. \quad (15)$$

The integrand now contains only the weakly singular $1/R$ kernel and is evaluated tetrahedron pair by tetrahedron pair.

3.2 Surface correction terms K^B , K^C , K^D

When the source basis $\mathbf{f}_n$ is a half-SWG function attached to a boundary face S_n , the inner integration by parts leaves a surface term from the boundary $\partial\Omega$:

$$\Phi_n(\mathbf{r}) \equiv \nabla \cdot \int_{\Omega} G(\mathbf{r}, \mathbf{r}') \mathbf{f}_n(\mathbf{r}') d^3r' = \int_{\Omega} G \nabla \cdot' \mathbf{f}_n d^3r' - \oint_{\partial\Omega} G(\mathbf{r}, \mathbf{r}') (\mathbf{f}_n \cdot \hat{\mathbf{n}}') d^2r'. \quad (16)$$

Similarly, the outer integration by parts against $\mathbf{f}_m$ produces a surface term if $\mathbf{f}_m$ is a half-SWG function on a boundary face S_m . The complete kernel is

$$K_{mn} = K_{mn}^A + K_{mn}^B + K_{mn}^C + K_{mn}^D, \quad (17)$$

with the three boundary contributions

$$K_{mn}^B = \int_{\Omega} \oint_{\partial\Omega} (\nabla \cdot \mathbf{f}_m)(\mathbf{r}) (\mathbf{f}_n \cdot \hat{\mathbf{n}}') G(\mathbf{r}, \mathbf{r}') d^2r' d^3r, \quad (18)$$

$$K_{mn}^C = \oint_{\partial\Omega} \int_{\Omega} (\mathbf{f}_m \cdot \hat{\mathbf{n}}) (\nabla \cdot' \mathbf{f}_n)(\mathbf{r}') G(\mathbf{r}, \mathbf{r}') d^3r' d^2r, \quad (19)$$

$$K_{mn}^D = - \oint_{\partial\Omega} \oint_{\partial\Omega} (\mathbf{f}_m \cdot \hat{\mathbf{n}}) (\mathbf{f}_n \cdot \hat{\mathbf{n}}') G(\mathbf{r}, \mathbf{r}') d^2r' d^2r. \quad (20)$$

By the half-SWG property $\mathbf{f}_n \cdot \hat{\mathbf{n}} = 1$ on S_n and zero on the remaining boundary faces, the surface integrals in (18)–(20) collapse to integrals over the specific faces S_m and S_n : K_{mn}^B is non-zero only when n is a boundary face, K_{mn}^C only when m is a boundary face, and K_{mn}^D only when both m and n are boundary faces.

3.3 Charge neutrality

The induced bound charge associated with a single half-SWG basis on a boundary face S_n is the sum of a bulk contribution $\rho_{\text{vol}} = -\kappa \nabla \cdot \mathbf{D}$ and a surface contribution $\sigma_{\text{surf}} = \kappa \mathbf{D} \cdot \hat{\mathbf{n}}$. Using (9) and the unit-flux property on S_n ,

$$Q_{\text{vol}} = - \int_{T_n^+} \kappa \nabla \cdot \mathbf{f}_n \, d^3r = -\kappa \frac{a_n}{V_n^+} \cdot V_n^+ = -\kappa a_n, \quad (21)$$

$$Q_{\text{surf}} = \int_{S_n} \kappa (\mathbf{f}_n \cdot \hat{\mathbf{n}}) \, d^2r = \kappa a_n. \quad (22)$$

The sum $Q_{\text{vol}} + Q_{\text{surf}} = 0$ holds exactly *only* when the surface contributions K^B, K^C, K^D are retained. Dropping them, as an internal-only SWG expansion does, leaves the particle with a spurious net bound charge $-\kappa a_n$ per boundary face and produces cross-section errors of order 10–30% on Mie tests. With the full decomposition (17), block-VIEM.jl reproduces the Mie sphere absorption cross section to $\sim 0.2\%$ at $\ell_c = 0.5$ (589 DoFs), an order-of-magnitude improvement per DoF over cubic-lattice DDA.

3.4 Mass matrix

The first term in (14) is the sparse mass matrix

$$M_{mn} = \int_{T_m \cap T_n} \mathbf{f}_m(\mathbf{r}) \cdot \mathbf{f}_n(\mathbf{r}) \, d^3r. \quad (23)$$

The integrand is a degree-2 polynomial in each tetrahedron, so the 5-point Gauss rule on the tetrahedron (degree 3 exact) evaluates M_{mn} without quadrature error.

4 Singular and Near-Singular Volume Integration

4.1 Generalised vertex Duffy transform

The weakly singular kernel $1/R$ in (15)–(20) remains integrable but standard Gauss quadrature fails when the source and test tetrahedra share a vertex, edge, or face, or when the outer Gauss point approaches a face of the inner element. Following Mousavi–Sukumar [4], all such cases are handled uniformly by a *vertex-only* Duffy transform applied to sub-tetrahedra obtained by projecting the singular point onto the inner element.

For $\alpha = 1$ (a $1/R$ singularity at a single vertex), the optimised transform from the unit cube $\mathbf{u} = (u_1, u_2, u_3) \in [0, 1]^3$ to barycentric coordinates $(\lambda_1, \lambda_2, \lambda_3, \lambda_4)$ with $\lambda_1 = 1$ at the singular vertex is [4, Eq. (1) with $\beta = 1$]

$$\lambda_1 = 1 - u_1, \quad \lambda_2 = u_1(1 - u_2), \quad \lambda_3 = u_1 u_2(1 - u_3), \quad \lambda_4 = u_1 u_2 u_3. \quad (24)$$

The face $u_1 = 0$ of the cube collapses entirely to the singular vertex, and $R \propto u_1$ to leading order along any ray departing from it. The total Jacobian combining (24) with the affine map from reference to physical coordinates is

$$J_D(\mathbf{u}) = 6 |V_{\text{tet}}| u_1^2 u_2. \quad (25)$$

The factor u_1^2 exactly cancels the $1/R$ singularity, leaving a smooth integrand on the unit cube that is integrated by a tensor-product Gauss–Legendre rule. For the K^A kernel (15) the recommended order is $5 \times 5 \times 5$ (125 points) for self and face-adjacent pairs, and a $4 \times 4 \times 4$ rule for edge- and vertex-adjacent pairs; a 5-point Gauss rule on the source tetrahedron is sufficient for all pairs that are not near the observation point.

4.2 Pair classification

Given an outer quadrature point $\mathbf{r}$ inside the test tetrahedron T_m , the source element T_n is classified and integrated according to the minimum distance:

- *self* ($T_n = T_m$): the outer Gauss point is the singular vertex. The source tetrahedron is subdivided into four sub-tetrahedra with the outer Gauss point as a common vertex, and each is integrated by the Duffy transform (24).
- *face-/edge-/vertex-adjacent*: the singular point is the nearest vertex, the nearest point on the shared edge, or the shared face. Sub-tetrahedra are generated as above around that singular point and integrated by the same Duffy rule.
- *far* (R larger than a chosen threshold): the inner integral is evaluated with a plain tetrahedron Gauss rule (5 points). Pairs further than the AIM near-field radius are never integrated directly (Sec. 5).

4.3 Surface integrals for K^B, K^C, K^D

The surface contributions in Sec. 3.2 require the integration of G over a triangle, optionally combined with an inner tetrahedron integral (K^B, K^C) or an outer triangle integral (K^D). We adopt a two-dimensional version of the Duffy transform on the reference triangle: from $(u_1, u_2) \in [0, 1]^2$ to barycentric coordinates (η_1, η_2, η_3) ,

$$\eta_1 = 1 - u_1, \quad \eta_2 = u_1(1 - u_2), \quad \eta_3 = u_1 u_2, \quad (26)$$

with Jacobian $J_{D,2}(\mathbf{u}) = 2|A_{\text{tri}}| u_1$. The linear factor u_1 cancels the $1/R$ singularity at the vertex $\eta_1 = 1$, and the face $u_1 = 0$ collapses to that vertex just as in the three-dimensional case. For the K^D self/edge/vertex branches, the outer triangle is first subdivided into sub-triangles at the singular feature and the 2D Duffy rule is applied to each.

The near-singular sub-cases of K^B and K^C (for example, the outer tetrahedron T_m having a face coincident with the inner boundary face S_n) are handled by projecting the outer Gauss point onto the inner triangle and applying the 2D Duffy rule around that projection.

5 FFT-Accelerated MVP: Adaptive Integral Method

Direct computation and storage of the dense impedance matrix Z scales as $\mathcal{O}(N_{\text{dof}}^2)$ in memory and $\mathcal{O}(N_{\text{dof}}^3)$ in operations for direct factorisation, quickly becoming prohibitive. The Adaptive Integral Method (AIM) [3] accelerates the matrix-vector product by representing the far-field part of the kernel on an auxiliary cubic grid and evaluating the resulting block-Toeplitz sum with a three-dimensional FFT, mirroring the Goodman–Draine–Flatau construction [9] used by `block-DDA-Py`.

5.1 Auxiliary grid and stencil

A Cartesian grid of spacing h_g is overlaid on the bounding box of the particle. Let $\mathbf{g}_p$ denote the position of grid point p . Each basis function $\mathbf{f}_n$ is associated with a stencil $\mathcal{S}_n$ of $M_{\text{st}} = (n_{\text{st}} + 1)^3$ nearby grid points surrounding its centroid. For a moment matching order n_{st} , the stencil is a cube with $n_{\text{st}} + 1$ grid points on each side.

5.2 Moment matching

For each basis function $\mathbf{f}_n$ and each stencil point $p \in \mathcal{S}_n$ we construct a projection weight $W_{np} \in \mathbb{C}^3$ so that the multipole moments of the point source at p match those of the continuous

source $\mathbf{f}_n$ up to order n_{st} . The moments are defined with respect to the basis centroid $\mathbf{c}_n$:

$$\mu_n^\alpha = \int_{\Omega} \mathbf{f}_n(\mathbf{r}) (\mathbf{r} - \mathbf{c}_n)^\alpha d^3r, \quad |\alpha| = \alpha_x + \alpha_y + \alpha_z \leq n_{\text{st}}, \quad (27)$$

and the weights $\{W_{np}\}_{p \in \mathcal{S}_n}$ are defined by the linear constraint

$$\sum_{p \in \mathcal{S}_n} W_{np} (\mathbf{g}_p - \mathbf{c}_n)^\alpha = \mu_n^\alpha, \quad \forall |\alpha| \leq n_{\text{st}}. \quad (28)$$

A companion set of weights matches the divergence moments $\int (\nabla \cdot \mathbf{f}_n)(\mathbf{r} - \mathbf{c}_n)^\alpha d^3r$ for the scalar potential part of the kernel. The linear system (28) is small (M_{st} unknowns per basis function, $\binom{n_{\text{st}}+3}{3}$ equations) and is solved once, during matrix setup, per basis function.

5.3 Far-field product via FFT

With the moment matching complete, the far-field part of the matrix-vector product $\mathbf{y}^{\text{far}} = K^{\text{far}} \mathbf{x}$ is evaluated in three steps:

1. *Projection.* For each basis function n and each stencil point p , accumulate $q_p = \sum_n W_{np} x_n$ onto the grid.
2. *Convolution.* Compute $Q_p = \sum_{p'} G(\mathbf{g}_p - \mathbf{g}_{p'}) q_{p'}$ via a three-dimensional FFT of q , point-wise multiplication by the precomputed FFT of G on the doubled grid, and an inverse FFT.
3. *Interpolation.* For each test function m , $y_m^{\text{far}} = \sum_{p \in \mathcal{S}_m} W_{mp}^\top Q_p$.

Because G depends only on the grid-point separation, its convolution matrix is block-Toeplitz and is computed with the 3D FFT of the same construction used in `block-DDA-Py` [9]. The cost is $\mathcal{O}(N_g \log N_g)$ per MVP, where N_g is the number of grid points. Memory for the grid Green function is also $\mathcal{O}(N_g)$ on the doubled domain.

5.4 Precorrection of near-field pairs

The FFT sum above is accurate only in the far field; for pairs with $\|\mathbf{c}_m - \mathbf{c}_n\| \leq r_{\text{near}}$ it is replaced by the exact integrals of Sec. 4. The construction is a *difference* of two quantities: for each near pair (m, n) we compute the exact K_{mn} and subtract from it the contribution that the FFT would have produced through the grid projection, storing the difference in a sparse precorrection matrix K^{pc} . The complete MVP is

$$\mathbf{y} = (D_M - \frac{\kappa}{\epsilon_{\text{bg}}}(K^{\text{far}} + K^{\text{pc}})) \mathbf{x}, \quad (29)$$

where D_M is the sparse diagonal mass contribution M/ϵ_p and K^{pc} has $\mathcal{O}(N_{\text{dof}})$ non-zero entries. The default near radius is $r_{\text{near}} = 2 \times (\text{stencil extent})$, verified in Phase-3 benchmarks to produce $< 0.5\%$ relative error between the AIM-accelerated and the dense matrix-vector products.

5.5 Surface-moment extension for K^B , K^C , K^D

The projection of Sec. 5.2 accelerates only the bulk-bulk kernel K^A of Eq. (15). The three boundary-face contributions K^B, K^C, K^D of Eqs. (18)–(20) are, in the Phase-1a release, stored as a separate sparse matrix `half_swg_extra` of dimension $N_{\text{bnd}} \times N_{\text{dof}}$, which for the iron-oxide doublet geometry dominates total live memory at 81–86% (measured at R/5, R/6 mesh refinement). This subsection extends the moment-matching projection to the boundary-face supports so that K^B, K^C, K^D share the same three-dimensional FFT convolution as K^A , reducing `half_swg_extra` to a sparse near-field correction block whose storage scales as $\mathcal{O}(N_{\text{bnd}})$ rather than $\mathcal{O}(N_{\text{bnd}} N_{\text{dof}})$. The construction follows Anastassiou *et al.* [6], which supplies the mathematical rigour (“thin-sheet constraint”) omitted from the original AIM presentation of Bleszynski *et al.* [5].

Scalar surface density on a volumetric grid

Inspection of Eqs. (18)–(20) shows that the three boundary kernels couple to a *single* scalar surface density

$$\sigma_n(\mathbf{r}) \equiv \mathbf{f}_n(\mathbf{r}) \cdot \hat{\mathbf{n}}(\mathbf{r}), \quad \mathbf{r} \in S_n, \quad (30)$$

supported on the single boundary triangle S_n of basis n ; by the half-SWG unit-flux property, $\sigma_n = 1$ on S_n and vanishes elsewhere on $\partial\Omega$. Because the divergence theorem of Sec. 3.2 has already contracted the vector basis $\mathbf{f}_n$ against the outward normal, the surface AIM projection requires neither Cartesian components nor a surface divergence of $\mathbf{f}_n$: only the scalar σ_n appears. This is a substantial simplification over the four-channel RWG projection of Anastassiou [6], where no preceding integration by parts is performed.

The triangle S_n is a two-dimensional manifold embedded in the three-dimensional grid $\{\mathbf{g}_p\}$. A naive surface moment $\int_{S_n} \sigma_n(\mathbf{r} - \mathbf{c}_n)^\alpha d^2r$ determines the grid projection only for multi-indices α with zero normal component; moments with $\alpha_\perp \geq 1$ (where $\perp$ denotes the local direction along $\hat{\mathbf{n}}$) remain unconstrained, and the projection is consequently rank-deficient on a volumetric grid. Following Anastassiou [6], we *thicken* σ_n to a volumetric distribution by a normal delta,

$$\tilde{\sigma}_n(\mathbf{r}) \equiv \sigma_n(\mathbf{r}_\parallel) \delta(\xi_n), \quad \xi_n = (\mathbf{r} - \mathbf{r}_{S_n}) \cdot \hat{\mathbf{n}}, \quad (31)$$

where $\mathbf{r}_\parallel$ is the tangential projection of $\mathbf{r}$ onto the plane of S_n and $\mathbf{r}_{S_n}$ is any reference point on S_n . The three-dimensional moments of $\tilde{\sigma}_n$ are then

$$\mu_n^{\alpha,S} \equiv \int_{\mathbb{R}^3} \tilde{\sigma}_n(\mathbf{r}) (\mathbf{r} - \mathbf{c}_n)^\alpha d^3r = \int_{S_n} \sigma_n(\mathbf{r}_s) (\mathbf{r}_s - \mathbf{c}_n)^\alpha d^2r_s, \quad (32)$$

where the right-hand side is evaluated in the global Cartesian basis so that components along $\hat{\mathbf{n}}$ with $\alpha_\perp \geq 1$ automatically vanish and enter the linear system (34) below as the “thin-sheet constraint”

$$\sum_{q \in \mathcal{S}_n^S} \Lambda_{nq}^S (\mathbf{g}_q - \mathbf{c}_n)^\alpha = 0 \quad \text{for all } \alpha \text{ with } \alpha_\perp \geq 1, \quad (33)$$

forcing the projected point sources to collapse to a zero-thickness sheet in the grid’s far-field representation.

Moment matching

In direct analogy with the bulk matching equation (28), the surface projection weights $\{\Lambda_{nq}^S\}_{q \in \mathcal{S}_n^S}$ are determined by

$$\sum_{q \in \mathcal{S}_n^S} \Lambda_{nq}^S (\mathbf{g}_q - \mathbf{c}_n)^\alpha = \mu_n^{\alpha,S}, \quad \forall |\alpha| \leq n_{\text{st}}, \quad (34)$$

where $\mathcal{S}_n^S$ is a cubic stencil of $M_{\text{st}} = (n_{\text{st}} + 1)^3$ grid nodes centred on the triangle centroid $\mathbf{c}_n$. The stencil size and grid are identical to the bulk case of Sec. 5.1, so the surface and bulk contributions share a single FFT plan. Only a single scalar linear system per boundary face must be solved; the Vandermonde-type matrix $[(\mathbf{g}_q - \mathbf{c}_n)^\alpha]$ is of size $\binom{n_{\text{st}}+3}{3} \times M_{\text{st}}$ and is independent of the kernel G , so it is factored once and reused for every boundary face.

Far-field product via the shared FFT

With the bulk vector weights $W_{np}^{\text{vec},\alpha}$ (three Cartesian components), the bulk divergence weights $W_{np}^{\nabla \cdot}$, and the new surface weights Λ_{nq}^S , the full far-field MVP $\mathbf{y}^{\text{far}} = K^{\text{far}} \mathbf{x}$ proceeds through a single grid pass. Define three source projections

$$q_p^{\text{vec},\alpha} = \sum_n W_{np}^{\text{vec},\alpha} x_n, \quad q_p^{\nabla \cdot} = \sum_n W_{np}^{\nabla \cdot} x_n, \quad q_p^S = \sum_{n \in \mathcal{N}_{\text{bnd}}} \Lambda_{np}^S x_n, \quad (35)$$

where $\mathcal{N}_{\text{bnd}}$ is the index set of half-SWG basis functions with a boundary face. A single three-dimensional FFT convolution with the Green-function kernel is then applied independently to each scalar source field,

$$Q_p^\bullet = \sum_{p'} G(\mathbf{g}_p - \mathbf{g}_{p'}) q_{p'}^\bullet, \quad \bullet \in \{\text{vec}, \alpha; \nabla \cdot; S\}, \quad (36)$$

reusing the precomputed FFT of G exactly as in Sec. 5.3. Finally, the test-side interpolation combines the vector and two scalar grid channels,

$$\begin{aligned} y_m^{\text{far}} = & k_0^2 \sum_{\alpha} \sum_{p \in S_m} W_{mp}^{\text{vec}, \alpha} Q_p^{\text{vec}, \alpha} + \sum_{p \in S_m} W_{mp}^{\nabla \cdot} (Q_p^S - Q_p^{\nabla \cdot}) \\ & + [m \in \mathcal{N}_{\text{bnd}}] \sum_{p \in S_m^S} \Lambda_{mp}^S (Q_p^{\nabla \cdot} - Q_p^S), \end{aligned} \quad (37)$$

where the first line accumulates $K^{A, \text{far}}$ and (via $Q^S - Q^{\nabla \cdot}$) $K^{B, \text{far}}$ minus the divergence-divergence piece of K^A , and the second line adds $K^{C, \text{far}}$ and $K^{D, \text{far}}$ for test functions on a boundary face (Iverson bracket $[\cdot]$). The symmetric appearance of $Q^{\nabla \cdot} - Q^S$ on both sides reflects the charge-neutrality identity $Q_{\text{vol}} + Q_{\text{surf}} = 0$ of Sec. 3.3: the bulk divergence and the boundary-face normal flux are the two halves of a single conserved charge distribution, and the AIM projection preserves this cancellation on the grid.

No additional FFT is required compared to the bulk-only pipeline of Sec. 5.3: the surface channel is absorbed as one extra scalar grid array, for a total of $3 + 2 = 5$ scalar FFTs per MVP (three vector components, one bulk divergence, and one surface density) versus $3 + 1 = 4$ in Phase-1a.

Near-field correction

For pairs (m, n) with at least one of $\{m, n\}$ on a boundary face and $\|\mathbf{c}_m - \mathbf{c}_n\| \leq r_{\text{near}}$, the far-field approximation in Eq. (37) is insufficient. In direct analogy with the bulk precorrection of Sec. 5.4, we store the difference between the Duffy-integrated $K_{mn}^{B/C/D}$ of Sec. 3.2 and its AIM-reconstructed far-field value in a sparse correction block $K^{S, \text{pc}}$. The storage scales as $\mathcal{O}(N_{\text{bnd}} \langle M_{\text{near}}^S \rangle)$, where $\langle M_{\text{near}}^S \rangle$ is the mean number of near-neighbours per boundary face. For a mesh with characteristic edge length ℓ_c and near radius $r_{\text{near}} = 2 \times (\text{stencil extent})$, this count is $\mathcal{O}(1)$, so $K^{S, \text{pc}} = \mathcal{O}(N_{\text{bnd}})$. On compact objects with $N_{\text{bnd}} = \mathcal{O}(N_{\text{dof}}^{2/3})$, this is a reduction from $\mathcal{O}(N_{\text{dof}}^{5/3})$ (`half_swg_extra` in Phase-1a) to $\mathcal{O}(N_{\text{dof}}^{2/3})$ —an asymptotic factor of N_{dof} .

Closed-form surface moment evaluation

The surface moment integral (32) over a planar triangle S_n admits a closed-form recursive evaluation in simplex coordinates, eliminating quadrature error entirely. Let $(\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3)$ be the triangle vertices, (ξ, η) the affine simplex coordinates of the unit isosceles triangle $T_0 = \{0 \leq \xi, \eta; \xi + \eta \leq 1\}$, so that $\mathbf{r}_s = \mathbf{r}_1 + \xi(\mathbf{r}_2 - \mathbf{r}_1) + \eta(\mathbf{r}_3 - \mathbf{r}_1)$. Each monomial factor $(r_\beta - c_{n, \beta})$, $\beta \in \{x, y, z\}$, becomes linear in (ξ, η) , $r_\beta - c_{n, \beta} = \mu_{10}^{(\beta)} \xi + \mu_{01}^{(\beta)} \eta + \mu_{00}^{(\beta)}$, and the order- Q = $|\boldsymbol{\alpha}|$ moment reduces to a bivariate polynomial integral

$$J_{\boldsymbol{\alpha}}^{(Q)} = \int_{T_0} \prod_{q=1}^Q (\mu_{10}^{(\beta_q)} \xi + \mu_{01}^{(\beta_q)} \eta + \mu_{00}^{(\beta_q)}) d\xi d\eta = \sum_{i=0}^Q \sum_{j=0}^Q \lambda_{ij}^{(Q)} \frac{i! j!}{(i+j+2)!}, \quad (38)$$

where β_q runs over the multi-index $\boldsymbol{\alpha}$ entry by entry (so Q factors appear in total). The polynomial coefficients $\lambda_{ij}^{(Q)}$ are generated by the recursion

$$\lambda_{ij}^{(Q+1)} = \lambda_{i-1, j}^{(Q)} \mu_{10}^{(\beta_{Q+1})} + \lambda_{i, j-1}^{(Q)} \mu_{01}^{(\beta_{Q+1})} + \lambda_{i, j}^{(Q)} \mu_{00}^{(\beta_{Q+1})}, \quad \lambda_{00}^{(0)} = 1, \quad (39)$$

with $\lambda_{ij}^{(Q)} = 0$ whenever $i < 0$ or $j < 0$. The final surface moment is recovered by multiplying $J_{\alpha}^{(Q)}$ by the Jacobian $2|S_n|$. The cost per boundary face is $\mathcal{O}(n_{\text{st}}^4)$ operations with no numerical quadrature, eliminating one of the two truncation-error sources in the Phase-1a projection (the bulk tetrahedral moments retain a Gauss-cubature error of $\sim 10^{-12}$ which dominates machine epsilon but is well below the AIM far-field truncation at $n_{\text{st}} = 2$).

Summary of Phase A impedance

With the surface projection in place, the complete AIM-accelerated impedance equation (29) is augmented to

$$\mathbf{y} = \left(D_M - \frac{\kappa}{\epsilon_{\text{bg}}} (K^{\text{far}} + K^{\text{pc}} + K^{S,\text{pc}})\right) \mathbf{x}, \quad (40)$$

where K^{far} now evaluates all four kernel contributions $K^A + K^B + K^C + K^D$ via Eq. (37) and $K^{S,\text{pc}}$ is the sparse surface-near-field correction. The Phase-1a `half_swg_extra` matrix is removed entirely.

Memory and timing scaling

The Au doublet benchmark ($R_{\text{monomer}} = 30$ nm, gap 3 nm, $\lambda_0 = 638$ nm, Johnson–Christy [15] gold) measured at four refinement levels $\ell_c = R/k$ for $k \in \{5, 6, 7, 8\}$ shows the total in-memory AIMOperator footprint growing as $\mathcal{O}(N_{\text{dof}}^{1.02})$ — essentially linear in N_{dof} , confirming the Phase A design target. The dominant contribution is the bulk precorrection block K^{pc} (69% to 72% of total), whose sparsity scales as $\mathcal{O}(N_{\text{dof}} n_{\text{near}})$ with $n_{\text{near}} \lesssim (M_{\text{st}} r_{\text{near}} / \ell_c)^3$ an $\mathcal{O}(1)$ count independent of N_{dof} . The new surface projection W^S adds 4% to 5% to the total, a small overhead compared with the eliminated $\mathcal{O}(N_{\text{dof}}^{5/3})$ Phase-1a `half_swg_extra` matrix. Setup and solve times also scale as $\mathcal{O}(N_{\text{dof}})$ within measurement noise, consistent with the AIM MVP cost $\mathcal{O}(N_g \log N_g + N_{\text{dof}} n_{\text{near}})$ at fixed wavelength. At $R/7$ the operator footprint is below 300 MiB and at $R/8$ below 450 MiB, so further refinement on a 16 GB workstation is feasible and is the next step toward closing the residual $\text{Im } S_{\text{fw},\theta}$ error at $\beta = \pi/2$ (Sec. 8.3). The full per-row table — DoF count, N_{bnd} , per-component memory, t_{setup} , t_{solve} — is in `docs/benchmark_results.md` §8.3.

Solvers

6.1 Direct and Krylov solvers

For small problems ($N_{\text{dof}} \lesssim$ a few thousand), the dense assembly of Z and its LU factorisation is practical and serves as a validation reference for the AIM path. For larger problems we use a Krylov iterative method (BiCGSTAB from `Krylov.jl` [12]) applied to the AIM operator (29). A Jacobi preconditioner formed from the diagonal of the sparse mass and precorrection contributions suffices in our tests.

6.2 Multi-orientation batch solve

Because the impedance matrix Z depends only on the particle geometry and permittivity, it can be assembled once and then reused across many orientations: each orientation changes only the right-hand side $\mathbf{b}$. The function `solve_cas_v2_orientations` is the user-facing entry point; by default it selects `method=:aim_bicgstab`, which builds the AIM operator (Sec. 5) once and then solves the block system $\mathbf{Z}\mathbf{X} = \mathbf{B}$ for all orientations simultaneously via a single block-Krylov iteration (Sec. 10). For small problems ($N_{\text{dof}} \lesssim 10^3$) `method=:dense` falls back to the textbook assemble- Z -and-LU-factorise path, back-substituting for every $(\alpha_\ell, \beta_\ell, \gamma_\ell)$ in the input list. For spheroids either route is especially efficient: only the $\alpha = 0$ orientation needs an independent solve for each β , and the remaining α values are obtained by the analytical expansion of Sec. 8.1.

6.3 Parallel execution

Every computationally significant loop in the setup, MVP, and post-processing paths is parallelised for shared-memory multi-core execution. The user controls the number of worker threads with `julia -t N` (or the `JULIA_NUM_THREADS` environment variable); no other configuration is required.

Setup. The precorrection assembly (Sec. 5.4) is the dominant setup cost for any realistic problem — $\gtrsim 90\%$ of `build_aim_operator` wall time at $N_{\text{dof}} \gtrsim 10^3$. Its outer near-column loop is partitioned into chunks and launched as independent `Threads.@spawn` tasks; each task accumulates its own COO triplets and the resulting `SparseMatrixCSC` is built once from the concatenated lists. The same pattern parallelises `build_aim_projection` (basis loop) and the near-field impedance assembly used by the dense path. BLAS is forced to a single thread inside the parallel region to avoid nested oversubscription from LAPACK calls (the monomial-matrix least-squares solves inside `build_aim_projection`). On an Intel Xeon 20-core machine against a $N_{\text{swg}} = 3041$ sphere mesh, `assemble_precorrection` drops from 25.9 s (1 thread) to 10.5 s (4 threads, $2.5\times$) and 8.9 s (10 threads, $2.9\times$).

Block MVP. The column loop in `aim_mvp(op, x)` is parallelised: each of the L right-hand-side columns is dispatched as its own task, so for a machine with N_{thr} Julia threads the FFT work of the block MVP runs at effective width $\min(L, N_{\text{thr}})$. Details in Sec. 10.

FFT. FFTW maintains its own thread pool; at module load `BlockVIEM` sets it to `Threads.nthreads()` via `FFTW.set_num_threads`. The exported `BlockVIEM.set_fft_threads(n)` resets it at run time. The recommended balance for a block MVP with L columns is

$$n_{\text{FFT}} = \max(1, \lceil N_{\text{thr}}/L \rceil), \quad (41)$$

so that the column and FFT parallelism together saturate the cores without oversubscription.

Parameter-sweep reuse. The projection matrices $\mathbf{W}_{x,y,z}$, $\mathbf{W}_{\text{div}}$, $\mathbf{W}_{\text{surf}}$ and the mass matrix $\mathbf{M}$ depend only on the mesh, the grid pitch and padding, and the polynomial order — not on k_0 or ϵ_p . The `build_aim_operator` constructor accepts keyword arguments `projection` and `mass` so that wavelength or material sweeps can build those pieces once and reuse them, rebuilding only the Green-function FFT kernel and the precorrection for each new (k_0, ϵ_p) . The MVP is bit-identical between the fresh and reused paths (tolerance $< 10^{-13}$).

7 Scattering Observables

Let D_n ($n = 1, \dots, N_{\text{dof}}$) denote the solution of (12) and $\mathbf{D}(\mathbf{r}) = \sum D_n \mathbf{f}_n(\mathbf{r})$ the reconstructed electric flux density. This section collects the observables computed from D_n that correspond to the CAS-v2 detector protocol [11] and can be compared directly with `block-DDA_Py`.

7.1 Far-field scattering amplitude

The scattered far field in the physics convention reads

$$\mathbf{E}^{\text{sca}}(\mathbf{r}) \sim \frac{e^{+ik_0 r}}{4\pi r} \mathbf{F}(\hat{\mathbf{r}}), \quad r \rightarrow \infty, \quad (42)$$

where the vector scattering amplitude is obtained by transverse projection of the Fourier transform of $\mathbf{D}$:

$$\mathbf{F}(\hat{\mathbf{r}}) = \frac{k_0^2 \kappa}{\epsilon_{\text{bg}}} (\mathbf{I}_3 - \hat{\mathbf{r}} \otimes \hat{\mathbf{r}}) \cdot \sum_{n=1}^{N_{\text{dof}}} D_n \int_{\Omega} \mathbf{f}_n(\mathbf{r}') e^{-ik_0 \hat{\mathbf{r}} \cdot \mathbf{r}'} d^3 r'. \quad (43)$$

The standard scattering amplitude used in Bohren–Huffman [10] is $\mathbf{f}(\hat{\mathbf{r}}) = \mathbf{F}(\hat{\mathbf{r}})/(4\pi)$.

7.2 Absorption cross section

The absorption cross section is evaluated directly from the volumetric Joule-loss formula, which is bilinear in $\mathbf{D}$ and hence reduces to a quadratic form in the expansion coefficients:

$$C_{\text{abs}} = \frac{k_0 \operatorname{Im}(\epsilon_p)}{\epsilon_{\text{bg}} |\epsilon_p|^2 |\mathbf{E}_0|^2} \operatorname{Re}(\mathbf{D}^\dagger \mathbf{M} \mathbf{D}), \quad (44)$$

where $\mathbf{M}$ is the mass matrix (23). For real ϵ_p this identically vanishes.

7.3 Scattering cross section by far-field integration

The scattering cross section is computed as the integral of the radiated flux density over the unit sphere,

$$C_{\text{sca}} = \frac{1}{16\pi^2 |\mathbf{E}_0|^2} \int_{S^2} |\mathbf{F}(\hat{\mathbf{r}})|^2 d\Omega, \quad (45)$$

using a product Gauss–Legendre (in $\cos \theta$) times trapezoid (in ϕ) rule with $2n_\theta^2$ directions. The default $n_\theta = 10$ is sufficient for size parameters $k_0 r_{ve} \lesssim 5$; larger targets use $n_\theta \geq 20$. The quadratic functional $|\mathbf{F}|^2$ is nonnegative and insensitive to the sign-convention subtleties that plague forward-direction optical-theorem estimates of C_{ext} , so in practice we obtain C_{ext} from

$$C_{\text{ext}} = C_{\text{abs}} + C_{\text{sca}}. \quad (46)$$

7.4 PCAS forward amplitude

The CAS-v2 forward scattering observables are defined with a circular incident polarisation [11]. Let $\hat{\mathbf{u}}_{\text{fw}}$ be the forward scattering direction (equal to the incident propagation direction) and $\hat{\mathbf{e}}_{\theta,\text{fw}}, \hat{\mathbf{e}}_{\phi,\text{fw}}$ the associated polar and azimuthal unit vectors in the particle frame. The s - and p -channel PCAS forward observables are

$$S_{\text{fw},\theta}^{\text{PCAS}} [= S_s(0)] = \frac{\sqrt{2}}{4\pi} \mathbf{F}(\hat{\mathbf{u}}_{\text{fw}}) \cdot \hat{\mathbf{e}}_{\theta,\text{fw}}, \quad (47)$$

$$S_{\text{fw},\phi}^{\text{PCAS}} [= S_p(0)] = -\frac{i\sqrt{2}}{4\pi} \mathbf{F}(\hat{\mathbf{u}}_{\text{fw}}) \cdot \hat{\mathbf{e}}_{\phi,\text{fw}}, \quad (48)$$

where the $\sqrt{2}$ and $i\sqrt{2}$ factors are inherited from the circular polarisation decomposition (6), and the $1/(4\pi)$ factor converts from $\mathbf{F}$ to the BH83-normalised scattering amplitude $\mathbf{f} = \mathbf{F}/(4\pi)$. Equations (47) and (48) are the two physical quantities measured by the PCAS instrument. For convenience, their arithmetic mean

$$S_{\text{fw},\text{mean}}^{\text{PCAS}} = \frac{S_{\text{fw},\theta}^{\text{PCAS}} + S_{\text{fw},\phi}^{\text{PCAS}}}{2} \quad (49)$$

is also stored in the `CASv2Result` container as a derived single-channel summary; (47) and (48) remain the primary observables.

7.5 OCBS backward amplitude

For the exact backward scattering direction $\hat{\mathbf{u}}_{\text{bk}} = -\hat{\mathbf{u}}_{\text{inc}}$, the laboratory basis is $\hat{\mathbf{e}}_{\theta,\text{bk}} = -\hat{\mathbf{e}}_{\theta,\text{inc}}$ and $\hat{\mathbf{e}}_{\phi,\text{bk}} = +\hat{\mathbf{e}}_{\phi,\text{inc}}$. The θ - and ϕ -channel backward amplitudes are defined in direct analogy

with (47)–(48) at $\hat{\mathbf{r}} = \hat{\mathbf{u}}_{\text{bk}}$, and the single observable reported by the OCBS (Optical Coherence Backscattering Sensor) instrument [11] is the linear combination

$$S_{\text{bk}}^{\text{OCBS}} = \frac{-S_{\text{bk},\theta}^{\text{OCBS}} + S_{\text{bk},\phi}^{\text{OCBS}}}{\sqrt{2}}. \quad (50)$$

As shown in [11], combining the two channels with this sign pattern cancels the off-diagonal Mishchenko elements $S_{12}(\pi) + S_{21}(\pi) = 0$ and produces a single, orientation-stable scalar observable.

8 Validation

The block-VIEM.jl solver has been validated against four independent references: the Mie series for homogeneous spheres (dielectric and plasmonic gold), an analytical α -symmetry identity for axisymmetric particles (Sec. 8.1), the multi-sphere T-matrix package MSTMFORCAS.JL [17] on a touching two-sphere doublet, and block-DDA.Py [11] on the spheroid, GRE ($\beta > 0$), and anisotropic- ε_p test cases. Sub-1% accuracy on all five CAS-v2 observables is reached at modest mesh sizes ($N \sim 2 \times 10^3$ half-SWG DoF) for the Mie reference, and agreement with the independent codes is at the few-percent level expected from the discretisation budget of either side. Mesh refinement on the plasmonic doublet recovers the theoretical linear-SWG convergence rate $p \approx 2$ on every observable.

The full set of validation conditions (geometry, material, mesh, solver settings), per-observable error tables (real and imaginary parts of $S_{\text{fw},\theta}$, $S_{\text{fw},\phi}$, S_{bk} , plus $C_{\text{ext/abs/sca}}$), the ℓ_c -refinement convergence-rate fits, and the AIM-operator memory scaling are collected in `docs/benchmark_results.md`; that document is the authoritative numerical record for citation in the paper. This section retains only the theoretical identity used to define the α -symmetry check (Sec. 8.1) and the asymptotic cost model used to reason about VIEM-vs-DDA per-DoF efficiency (Sec. 8.2); their associated numerical tables live in the benchmark document.

8.1 Oblate spheroid α -symmetry

A prolate or oblate spheroid has a continuous C_∞ rotation symmetry about its figure axis. When the orientation is parameterised by intrinsic ZYZ Euler angles with $\gamma = 0$, rotating the first angle α does not change the physical orientation of an axisymmetric particle in the particle frame; it only rotates the incident polarisation basis by α about $\hat{\mathbf{u}}_{\text{inc}}$. For the CAS-v2 observables this implies the analytical α -expansion [11]

$$S_\theta(\alpha, \beta, \gamma=0) = A(\beta) + B(\beta) e^{+2i\alpha}, \quad (51)$$

$$S_\phi(\alpha, \beta, \gamma=0) = A(\beta) - B(\beta) e^{+2i\alpha}, \quad (52)$$

with

$$A(\beta) = \frac{1}{2} [S_\theta(0, \beta, 0) + S_\phi(0, \beta, 0)], \quad B(\beta) = \frac{1}{2} [S_\theta(0, \beta, 0) - S_\phi(0, \beta, 0)]. \quad (53)$$

The identity (51)–(52) is the exact basis of the α -analytical expansion that block-DDA.Py uses to reduce the orientation sweep to solves at $\alpha = 0$ only. Because VIEM after the physics-convention switch lives in the same time-convention as block-DDA.Py, the sign of the exponent is $+2i\alpha$ — identical to block-DDA.Py.

The benchmark script `benchmarks/cas_v2/spheroid_ar3.jl` verifies (51)–(52) numerically by solving an oblate spheroid (BC_RATIO = 3) at $\beta = \pi/4$ for $\alpha \in \{0, \pi/8, \pi/4, 3\pi/8, \pi/2\}$ and comparing each independently solved amplitude with the analytical prediction; the residual is at the level of double-precision machine error (see `docs/benchmark_results.md` §3). The α -symmetry check is internally consistent only and does not confirm absolute amplitudes; that is done by direct cross-validation against block-DDA.Py on the same spheroid, on GRE shapes with $\beta > 0$, and on anisotropic ε_p spheres (`docs/benchmark_results.md` §§4–6).

8.2 Accuracy and computational cost vs. block-DDA_Py

On the homogeneous Mie sphere ($m_p = 1.5 + 0.01i$, $\lambda_0 = 10\ \mu\text{m}$, $r = 1\ \mu\text{m}$, $x \approx 0.63$) block-VIEM.jl is roughly **10–12× more accurate per DoF** than block-DDA_Py: matching the $\sim 1.25\%$ DDA C_{abs} error obtained at ~ 4500 dipoles requires only ~ 400 half-SWG DoFs, and at $N \sim 8000$ the VIEM error is an order of magnitude smaller. The full per-DoF accuracy table and the high-contrast $m_p = 3.17 + 0.16i$ extension are tabulated in `docs/benchmark.results.md` §9.

There are four mutually reinforcing reasons.

1. *Conforming geometry.* A tetrahedral mesh represents curved boundaries to $O(\ell_c^2)$, whereas a cubic lattice stair-cases them to $O(d)$. For Mie spheres this alone removes a geometric contribution to C_{abs} and C_{sca} that scales as d rather than d^2 .
2. *Conforming vector space.* The SWG/half-SWG basis is $H(\text{div})$ -conforming, so the normal component of $\mathbf{D}$ is automatically continuous across tetrahedron faces — the physically correct boundary condition at a dielectric interface. Expanding $\mathbf{D}$ instead of $\mathbf{E}$ [1, Sec. II.B] further ensures that the polarisation jump at $\partial\Omega$ is represented exactly by the half-SWG functions on the boundary faces, with the charge-neutrality identity of Sec. 3.3. In DDA the induced polarisation is piecewise constant at the dipole centres, so the interface condition is only recovered in the limit $d \rightarrow 0$.
3. *Linear vector basis vs point dipoles.* Within each tetrahedron $\mathbf{D}$ is represented by up to four SWG functions, each linear in position and with a piecewise-constant divergence. DDA represents the polarisation by a single point dipole per element, i.e. a delta distribution localised at the lattice node. The tetrahedral moment approximation is therefore intrinsically one order higher, and in the far field the error per element scales as $k_0 h$ rather than $(k_0 d)^0$.
4. *Physical polarisability.* DDA requires a dynamic polarisability prescription (Clausius–Mossotti plus the $(2/3)(ka)^3$ radiative correction [8]) to cancel leading-order interaction errors, and the choice of prescription feeds back into the error. VIEM solves the integral equation directly; $\mathbf{D}$ is the physical variable, and no such tuning is required.

Against these accuracy advantages, block-VIEM.jl pays a higher per-DoF setup cost. Table 1 summarises the asymptotic cost and memory for a matrix-free MVP:

Table 1: Asymptotic cost model. N is the number of unknowns, N_g the number of AIM or FFT grid points (typically $N_g \sim 2N$ for AIM), n_{st} the AIM moment matching order, n_{it} the number of Krylov iterations, and L the number of particle orientations in a batch solve.

Quantity	block-DDA_Py	block-VIEM.jl (half-SWG + AIM)
Element geometry	cubic lattice	unstructured tetrahedra
Green kernel on grid	analytic dyadic, $\mathcal{O}(N_g)$	scalar + moment-matched dyadic, $\mathcal{O}(N_g)$
Near-field “precorrection”	none (exact on the lattice)	Duffy volumetric, $\mathcal{O}(N n_{\text{near}})$
MVP cost	$\mathcal{O}(N_g \log N_g)$	$\mathcal{O}(N_g \log N_g + N n_{\text{near}})$
Multi-orientation batch	block-BiCGSTAB on L RHS	shared LU (small N) or L indep. BiCGSTAB
Dominant memory	$\mathcal{O}(N_g)$ FFT kernel	$\mathcal{O}(N_g)$ FFT kernel + $\mathcal{O}(N n_{\text{near}})$ precorr.

The central difference is that DDA has a strictly translation-invariant interaction matrix, so a single FFT on the doubled lattice captures the entire MVP exactly. VIEM has no such

property: each basis function sees a different discrete support of neighbouring tetrahedra, and the AIM moment matching is only accurate for pairs whose separation is large compared with the element size. Near pairs must therefore be computed directly via Duffy quadrature and stored as a sparse precorrection matrix (Sec. 5.4). For typical settings, this near-field work dominates both the matrix assembly time and the per-iteration MVP cost at small to moderate N ; only for very large meshes does the $N_g \log N_g$ FFT cost become comparable.

The overall trade-off is therefore: **VIEM spends more work per unknown (near-field Duffy integrals and precorrection), but uses an order of magnitude fewer unknowns to reach a given accuracy for high-contrast particles with curved boundaries.** For sphere-like, low-contrast problems DDA’s simplicity wins on wall-clock time; for the high-contrast iron-oxide aggregates and faceted gold nanoparticles targeted by this project, VIEM is the clear winner — the $\ell_c = 0.50 \mu\text{m}$ row in `docs/benchmark_results.md` §9.3 (589 DoF, 0.21% C_{abs} error against the Mie reference) is representative of production use.

8.3 Two-sphere doublet vs MSTM (multi-orientation)

A second independent cross-validation compares block-VIEM.jl against the Multi-Sphere T-Matrix package MSTMFORCAS.JL [17], which produces numerically exact CAS-v2 observables for aggregates of homogeneous spheres. The benchmark target is a touching doublet of equal spheres (radius $0.030 \mu\text{m}$, axial gap $0.003 \mu\text{m}$) at $\lambda_0 = 0.638 \mu\text{m}$, three orientations $\beta \in \{0, \pi/4, \pi/2\}$, and two materials: a high-contrast dielectric ($m_p = 1.60 + 0.01i$, polystyrene-like) and plasmonic gold ($m_p = 0.17525 + 3.4830i$, Johnson & Christy 1972). The half-SWG VIEM mesh uses $\ell_c = R/5$ ($N_{\text{dof}} = 11\,501$) solved by AIM block-BiCGSTAB at $\text{tol} = 10^{-7}$; the MSTM reference uses VSWF truncation forced to $N = 15$ ($\text{rtol} \leq 10^{-6}$ on both materials at $x \approx 0.30$). For the dielectric doublet $|S_{\text{fw,mean}}|$ matches MSTM to $\leq 1.6\%$ across all orientations. For the plasmonic Au doublet the error grows with β from 1.3% at $\beta = 0$ to 4.0% at $\beta = \pi/2$, with strong polarisation anisotropy ($\text{Im } S_{\text{fw},\theta}$ at $\beta = \pi/2$ reaches 10.5%) traced to the concentration of the plasmonic surface mode in the sub-wavelength inter-sphere gap when the doublet axis is aligned with the incident polarisation.

A four-point mesh-refinement study at $\ell_c = R/k$ for $k \in \{5, 6, 7, 8\}$, fitted as $\text{rel. err.}(\ell_c) \propto (\ell_c/R)^p$, recovers the theoretical linear-SWG rate $p \approx 2.10\text{--}2.49$ on every component, including the stiff $\text{Im } S_{\text{fw},\theta}$ at $p = 2.10$. An earlier two-point fit using only $R/5$ and $R/6$ had given $p \in [0.78, 3.39]$; that spread is now understood as a two-point artefact. Phase A (Sec. 5.5) brings the AIM-operator memory to a near-linear $\mathcal{O}(N_{\text{dof}}^{1.02})$ scaling, so the $R/8$ ($N_{\text{dof}} = 45\,585$) refinement fits in 413 MiB of operator memory and is the basis for the extrapolation that $\ell_c = R/10$ should bring $\text{Im } S_{\text{fw},\theta}$ below 3% .

MSTMforCAS numerical-stability fixes. MSTMforCAS $\geq 0.4.3$ incorporates three numerical-stability fixes contributed as part of this benchmark work, *without which the $N = 15$ reference cannot be computed at $x \approx 0.3$* (the package previously failed catastrophically beyond $N = 6$):

1. Miller’s downward recurrence for the Riccati–Bessel $\psi_n(x)$ replaces the conditionally-stable upward recurrence (Wiscombe 1980). At $x \approx 0.3$ the upward recurrence amplifies a spurious χ_n component by $(2n-1)/x \approx 44$ per step while the physical $\psi_n(x) \sim x^n/(2n-1)!!$ underflows to $\sim 10^{-9}$ at $n = 8$, drowning the signal in numerical noise.
2. Correct sizing of the per-sphere Mie-coefficient buffer `mie_vecs[i]` to `nois[i]` when the user-requested `truncation_order` exceeds the default Wiscombe bound `mie_nmax(x)`.
3. Miller’s downward recurrence for the spherical-Bessel $j_n(z)$ used in the multi-sphere translation operator, eliminating the analogous $y_n(kd) \sim (2n-1)!!(kd)^{-n-1}$ precision loss at small kd that previously destabilised $N \geq 7$.

For aggregates of plasmonic monomers the appropriate MSTM truncation is set by the *inter-sphere coupling* amplified by $|m \cdot x|$, not by the single-sphere Mie convergence. MSTM reference data should always be supplied with an explicit N_{trunc} and a per-point convergence check (`benchmarks/cas_v2/doublet_mstm/check_trunc_convergence.jl`).

The full benchmark setup, the per-orientation Re/Im-resolved error tables for $S_{\text{fw},\theta}$ and $S_{\text{fw},\phi}$, the ℓ_c -refinement convergence-rate fits, and the Phase A AIM-operator memory measurements are tabulated in `docs/benchmark_results.md` §§7–8.

9 Spheroid Sweep HDF5 Output

For parameter-sweep applications the forward scattering amplitudes on a five-dimensional grid are written to an HDF5 file whose layout is deliberately identical to that produced by `block-DDA-Py` [11], so the downstream consumer `PCAS_Bayes_APM_Nonspherical/lut_generation/build_spheroid_lut.py` reads VIEM output without modification. The schema is documented in `block-DDA-Py/.claude/spheroid_h5_schema_spec.md` and summarised here for completeness.

9.1 Parameter grid

The five grid axes are stored at the root level of the file and shared across wavelength groups:

- `D_ve_grid` — volume-equivalent diameter D_{ve} [μm];
- `RI_real_grid` — real part of the particle refractive index, $\text{Re}(m_p)$;
- `log_AR_grid` — base-10 logarithm of $\text{bc_ratio} = b/c$, following the `block-DDA-Py` convention where $\text{bc_ratio} > 1$ is oblate, < 1 is prolate, and $= 1$ is a sphere;
- `cos_theta_o_half_grid` — $\cos \theta_o$ on $[0, 1]$ (half-domain, extended to $[-1, 1]$ by the consumer using the $\theta_o \rightarrow \pi - \theta_o$ mirror symmetry);
- `phi_o_grid` — ϕ_o on $[0, \pi]$ (half-domain, extended to $[0, 2\pi]$ by the consumer using the ϕ_o -periodicity $S(\theta_o, \phi_o + \pi) = S(\theta_o, \phi_o)$).

All axes must be equidistant; `write_spheroid_sweep_h5` enforces this constraint and raises an error otherwise, because the downstream `NdimSpline_JAX` consumer assumes uniform grids.

9.2 Per-wavelength datasets

Results for each wavelength λ_0 are stored under a group `wl.<value>` (e.g. `wl_0p638` for $\lambda_0 = 0.638 \mu\text{m}$), with the attributes `wl_0` and `m_m`. Each group contains

- `S_fw_theta_re`, `S_fw_theta_im`, `S_fw_phi_re`, `S_fw_phi_im`: the real and imaginary parts of $S_{\text{fw},\theta}$ and $S_{\text{fw},\phi}$ on the shape $(N_{D_{ve}}, N_{RI}, N_{AR}, N_{u_{1/2}}, N_{\phi_o})$;
- `converged`: a boolean array of shape $(N_{D_{ve}}, N_{RI}, N_{AR})$ indicating successful solves. Non-converged cells carry `NaN+NaNi` in the four amplitude arrays across all $(N_{u_{1/2}}, N_{\phi_o})$ entries.

The ϕ_o axis corresponds exactly to the Euler angle α , and the populating code applies the analytical α -expansion (51)–(52) to the $\alpha = 0$ solves, so only $N_{u_{1/2}}$ independent VIEM solves are required per $(D_{ve}, \text{Re}(m_p), \text{AR})$ triple.

10 AIM-accelerated block-Krylov multi-orientation solve

For large orientation sweeps — orientation LUTs typically require $L = 10^2$ – 10^3 incidence directions per $(D_{ve}, \text{Re}(m_p), \text{AR})$ cell — reusing the same LU factorization of the dense impedance matrix (Sec. ?? style) becomes infeasible as soon as $N \gg 10^3$. For those problems `block-VIEM.jl` v0.2.0 exposes an AIM-accelerated *block-Krylov* path that solves

$$\mathbf{Z} \mathbf{X} = \mathbf{B}, \quad \mathbf{X}, \mathbf{B} \in \mathbb{C}^{N \times L}, \quad (54)$$

with a single pre-built AIM operator and a single block iteration that acts on all L right-hand sides simultaneously.

Block MVP. The AIM operator supports the multi-RHS form

$$\text{aim_mvp}(\mathbf{X}) = \frac{1}{\epsilon_p} \mathbf{M} \mathbf{X} - \frac{\kappa}{\epsilon_{bg}} [\mathbf{K}_{\text{AIM}}(\mathbf{X}) + \mathbf{K}_{\text{near}} \mathbf{X}] + \mathbf{K}_{\text{half-SWG}} \mathbf{X},$$

where $\mathbf{M}, \mathbf{K}_{\text{near}}, \mathbf{K}_{\text{half-SWG}}$ are sparse and their matrix-matrix products cost $\mathcal{O}(\text{nnz} \cdot L)$ in a single BLAS call. The far-field convolution $\mathbf{K}_{\text{AIM}}(\mathbf{X})$ is applied column-by-column and the L columns are dispatched across the available Julia threads via `Threads.@spawn`; the FFT plans themselves are shared and use the process-wide FFTW thread pool, so the three levels of parallelism — column, FFT, and iteration — can be balanced as $n_{\text{col}} \times n_{\text{FFT}} \approx n_{\text{cores}}$ via `BlockVIEM.set_fft_threads`. For a single 20-core machine at $L = 8$ we measure a $2.6\times$ per-column speed-up over the serial baseline.

Block BiCGSTAB. The default solver follows Tadano, Sakurai and Kuramashi [13] and is a direct port of `block-DDA.py`'s `bl_bicgstab_jacobi_mvp_fft`, sans the Jacobi preconditioner (which is cheap for DDA because the dipole self-term is diagonal, but would require $\mathcal{O}(N)$ FFT probes for VIEM; the SWG mass matrix already normalizes the system adequately). Given $\mathbf{B} \in \mathbb{C}^{N \times L}$, set $\mathbf{X}_0 = \mathbf{0}$, $\mathbf{R}_0 = \mathbf{B}$, $\mathbf{P} = \mathbf{R}_0$, and $\tilde{\mathbf{R}}_0 = \mathbf{R}_0$, then iterate

$$\mathbf{V}_k = \mathbf{A} \mathbf{P}_k, \quad (55)$$

$$\boldsymbol{\alpha}_k = (\tilde{\mathbf{R}}_0^H \mathbf{V}_k)^{-1} (\tilde{\mathbf{R}}_0^H \mathbf{R}_k), \quad (56)$$

$$\mathbf{T}_k = \mathbf{R}_k - \mathbf{V}_k \boldsymbol{\alpha}_k, \quad (57)$$

$$\mathbf{Z}_k = \mathbf{A} \mathbf{T}_k, \quad (58)$$

$$\xi_k = \frac{\text{tr}(\mathbf{Z}_k^H \mathbf{T}_k)}{\text{tr}(\mathbf{Z}_k^H \mathbf{Z}_k)}, \quad (59)$$

$$\mathbf{X}_{k+1} = \mathbf{X}_k + \mathbf{P}_k \boldsymbol{\alpha}_k + \xi_k \mathbf{T}_k, \quad (60)$$

$$\mathbf{R}_{k+1} = \mathbf{T}_k - \xi_k \mathbf{Z}_k, \quad (61)$$

$$\boldsymbol{\beta}_k = -(\tilde{\mathbf{R}}_0^H \mathbf{V}_k)^{-1} (\tilde{\mathbf{R}}_0^H \mathbf{Z}_k), \quad (62)$$

$$\mathbf{P}_{k+1} = \mathbf{R}_{k+1} + (\mathbf{P}_k - \xi_k \mathbf{V}_k) \boldsymbol{\beta}_k, \quad (63)$$

stopping once $\|\mathbf{R}_{k+1}\|_F / \|\mathbf{B}\|_F < \text{tol}$. Each outer iteration performs exactly two block MVPs ($\mathbf{A} \mathbf{P}$ and $\mathbf{A} \mathbf{T}$) and four $L \times L$ Gram matrices, so the per-iteration cost is $\mathcal{O}(T_{\text{MVP}} \cdot L + NL^2)$. Block BiCGSTAB can break down when the columns of $\mathbf{R}_k$ lose numerical independence; in practice this is rare for the physically distinct incidence directions of a CAS-v2 LUT, but the fallback below exists.

Block GMRES. The fallback is unrestarted Block GMRES [14]: a block-Arnoldi recurrence with thin QR factorization of each new block Krylov vector builds an orthonormal basis $\mathbf{V}_1, \mathbf{V}_2, \dots$ of the block Krylov space $\mathcal{K}_k(\mathbf{A}; \mathbf{B})$, together with an upper block-Hessenberg matrix

$\underline{\mathbf{H}}_k$. At iteration k the approximate solution is $\mathbf{X}_k = \mathbf{X}_0 + [\mathbf{V}_1 | \cdots | \mathbf{V}_k] \mathbf{Y}_k$, where $\mathbf{Y}_k$ minimizes $\|\underline{\mathbf{H}}_k \mathbf{Y} - \mathbf{E}_1 \mathbf{\Lambda}\|_F$ with $\mathbf{\Lambda}$ the thin-QR factor of the initial residual. Unlike BiCGSTAB the memory grows linearly in the iteration count (k blocks of shape $N \times L$ are retained), so in practice Block GMRES is used as a robust fallback rather than a default.

Numerical verification. On an oblate AR= 3 spheroid ($D_{ve} = 0.40 \mu\text{m}$, $m_p = 1.5$, $\lambda_0 = 0.638 \mu\text{m}$, $N = 2009$ half-SWG DoFs) the two block solvers reach tolerance 10^{-8} in roughly 50 block iterations on $L = 5$ orientations, and their CAS-v2 forward amplitudes agree to $\sim 10^{-10}$ relative. Compared to the dense LU reference, the AIM block-Krylov observables agree to $\sim 0.8\%$, which is the AIM discretization error at the chosen pitch and not a Krylov convergence issue. The analytical $+2j\alpha$ spheroid symmetry (Sec. ??) on the AIM block-BiCGSTAB solution holds at $\sim 5 \times 10^{-10}$, confirming that the block iteration preserves orientation symmetry to the solver tolerance.

11 Particle shape models and cross-validation

The two particle-shape families supported by `block-VIEM.jl` are the Gaussian Random Ellipsoid (GRE) parameterised by $(r_{v,\text{base}}, b/c, a/b, \beta)$ following Muinonen & Pieniluoma [16], and the sphere-aggregate model represented by a `SphereAggregate` container of monomer centres and radii. Both pipelines emit a single conforming tetrahedral mesh, share the same volume-preserving final rescale (so the discretised volume matches the analytical target to machine precision, in parity with `block-DDA_Py`'s `Target...init...` spacing adjustment), and feed the SWG / half-SWG basis of Sec. 2.3. The full specification of parameter ranges, the `neck_ratio` neck-width convention, the equal-radius polydisperse extension, the adaptive characteristic length, and worked discretisation examples is given in `docs/descriptions_particle_shape_model.md` and is not reproduced here.

Cross-validation against `block-DDA_Py` is performed on three deformed GRE configurations with $\beta = 0.10, 0.15, 0.20$ and `BC_RATIO/AB_RATIO` in the range $[1.0, 3.0]$ at $\lambda_0 = 0.638 \mu\text{m}$, $m_p = 1.5 + 0.01i$, $r_{v,\text{base}} = 0.20 \mu\text{m}$, two tilt angles $\beta_{\text{ori}} = \pi/4, \pi/2$. The *same* Gaussian random deformation field $s(\theta, \phi)$ is generated by the Python driver and consumed by the Julia driver via `gre_mesh_with_field` so both codes solve the identical shape. Relative errors on the complex forward amplitudes are 0.1–2.6 %, consistent with the discretisation-error level observed in the smooth-spheroid benchmark (Sec. 8.2); per-case tables are in `docs/benchmark_results.md` §5.

12 Anisotropic Permittivity

Version 0.4.0 extends the EFVIE-D formulation to diagonal-tensor permittivity $\bar{\epsilon}_p = \text{diag}(\epsilon_x, \epsilon_y, \epsilon_z)$ in the particle frame, matching the `m_p_xyz` parameter of `block-DDA_Py`.

12.1 Modified weak form

Defining the component-wise contrast $\kappa_\alpha = (\epsilon_\alpha - \epsilon_{\text{bg}})/\epsilon_\alpha$ and its average $\bar{\kappa} = (\kappa_x + \kappa_y + \kappa_z)/3$, the Galerkin impedance element becomes

$$Z_{mn} = \sum_\alpha \frac{1}{\epsilon_\alpha} \int (f_m)_\alpha (f_n)_\alpha dV - \frac{1}{\epsilon_{\text{bg}}} \iint \left[k_0^2 \sum_\alpha \kappa_\alpha (f_m)_\alpha (f_n)_\alpha - \bar{\kappa} (\nabla \cdot \mathbf{f}_m) (\nabla' \cdot \mathbf{f}_n) \right] G dV' dV. \quad (64)$$

The simplification $\nabla' \cdot (\bar{\kappa} \cdot \mathbf{f}_n) = \bar{\kappa} \nabla' \cdot \mathbf{f}_n$ exploits the SWG structure $\partial(f_n)_\alpha / \partial r_\alpha = a_n / (3V)$ (identical for all α).

12.2 Implementation

- **Mass term:** component-weighted inner product $\sum_{\alpha} (1/\epsilon_{\alpha}) \int (f_m)_{\alpha} (f_n)_{\alpha} dV$.
- **Radiation kernel (bulk):** k_0^2 term uses per-component κ_{α} ; $\nabla \nabla \cdot$ term uses $\bar{\kappa}$.
- **Half-SWG surface terms** (K^B , K^C , K^D): scaled by $\bar{\kappa}$ (exact for isotropic; $O(|\Delta\kappa|/|\kappa|)$ approximation for anisotropic).
- **AIM FFT-MVP:** per-channel κ_{α} weighting on the x, y, z vector channels; $\bar{\kappa}$ on the divergence channel.
- **Far-field amplitude:** $\mathbf{F} = (k_0^2/\epsilon_{\text{bg}})(\mathbf{I} - \hat{\mathbf{r}}\hat{\mathbf{r}}) \cdot (\bar{\kappa} \cdot \mathbf{P})$.
- **Absorption:** $C_{\text{abs}} = \frac{k_0}{\epsilon_{\text{bg}}} \sum_{\alpha} \frac{\text{Im}(\epsilon_{\alpha})}{|\epsilon_{\alpha}|^2} \int |D_{\alpha}|^2 dV$.

When $\epsilon_x = \epsilon_y = \epsilon_z$ all expressions reduce to the isotropic formulation of Sec. ?? and earlier sections.

12.3 Cross-validation vs block-DDA_Py

Cross-validation against block-DDA_Py is performed on a sphere ($r_{ve} = 0.20 \mu\text{m}$, $\lambda_0 = 0.638 \mu\text{m}$, $n_m = 1$, $\ell_c = 0.035 \mu\text{m}$, $N_{\text{VIEM}} \approx 8000$ half-SWG DoFs) with three diagonal-tensor configurations $\mathbf{m}_p = (m_x, m_y, m_z)$ ranging from isotropic $[1.5, 1.5, 1.5]$ to strongly anisotropic $[1.6, 1.5, 1.4]$, at tilt angles $\beta_{\text{ori}} = \pi/4, \pi/2$. The isotropic baseline matches the existing spheroid benchmark accuracy (~ 1.3 – 2.4%); errors grow moderately with anisotropy strength, reaching $\sim 5\%$ for the “strong” case at $\beta_{\text{ori}} = \pi/2$, dominated by the $\bar{\kappa}$ approximation in the half-SWG surface terms and the cubic-vs-tetrahedral discretisation mismatch. Per-case tables are in `docs/benchmark_results.md` §6.

References

- [1] D. H. Schaubert, D. R. Wilton, and A. W. Glisson, “A tetrahedral modeling method for electromagnetic scattering by arbitrarily shaped inhomogeneous dielectric bodies,” *IEEE Trans. Antennas Propag.* **32**(1), 77–85 (1984).
- [2] J. L. Volakis and K. Sertel, *Integral Equation Methods for Electromagnetics* (SciTech Publishing, 2012).
- [3] X.-Q. Sheng and W. Song, *Essentials of Computational Electromagnetics* (Wiley-IEEE, 2012).
- [4] S. E. Mousavi and N. Sukumar, “Generalized Duffy transformation for integrating vertex singularities,” *Comput. Mech.* **45**(2-3), 127–140 (2010). <https://doi.org/10.1007/s00466-009-0424-1>.
- [5] E. Bleszynski, M. Bleszynski, and T. Jaroszewicz, “AIM: Adaptive integral method for solving large-scale electromagnetic scattering and radiation problems,” *Radio Sci.* **31**(5), 1225–1251 (1996). <https://doi.org/10.1029/96RS02504>.
- [6] H. T. Anastassiou, M. Smelyanskiy, S. Bindiganavale, and J. L. Volakis, “Scattering from relatively flat surfaces using the adaptive integral method,” *Radio Sci.* **33**(1), 7–16 (1998). <https://doi.org/10.1029/97RS02104>.
- [7] E. M. Purcell and C. R. Pennypacker, “Scattering and absorption of light by nonspherical dielectric grains,” *Astrophys. J.* **186**, 705–714 (1973).

- [8] B. T. Draine, “The discrete-dipole approximation and its application to interstellar graphite grains,” *Astrophys. J.* **333**, 848–872 (1988).
- [9] J. J. Goodman, B. T. Draine, and P. J. Flatau, “Application of fast-Fourier-transform techniques to the discrete-dipole approximation,” *Opt. Lett.* **16**(15), 1198–1200 (1991).
- [10] C. F. Bohren and D. R. Huffman, *Absorption and Scattering of Light by Small Particles* (Wiley, New York, 1983).
- [11] N. Moteki and K. Adachi, “Measuring the polarized complex forward-scattering amplitudes of single particles in unbounded fluid flow: CAS-v2 protocol,” *Opt. Express* **32**(21), 36500–36522 (2024). <https://doi.org/10.1364/OE.533776>.
- [12] A. Montoison and D. Orban, “Krylov.jl: A Julia basket of hand-picked Krylov methods,” *J. Open Source Softw.* **8**(89), 5187 (2023). <https://doi.org/10.21105/joss.05187>.
- [13] H. Tadano, T. Sakurai, and Y. Kuramashi, “Block BiCGSTAB algorithms for solving linear systems with multiple right-hand sides,” *JSIAM Letters* **1**, 44–47 (2009).
- [14] V. Simoncini and D. B. Szyld, “Flexible inner-outer Krylov subspace methods” / “Block GMRES,” *Numer. Linear Algebra Appl.* **2**(6), 539–560 (1996).
- [15] P. B. Johnson and R. W. Christy, “Optical constants of the noble metals,” *Phys. Rev. B* **6**(12), 4370–4379 (1972). <https://doi.org/10.1103/PhysRevB.6.4370>.
- [16] K. Muinonen and T. Pieniluoma, “Light scattering by Gaussian random ellipsoid particles: First results with discrete-dipole approximation,” *J. Quant. Spectrosc. Radiat. Transfer* **112**(11), 1747–1752 (2011). <https://doi.org/10.1016/j.jqsrt.2011.02.001>.
- [17] N. Moteki, *MSTMforCAS.jl – Julia implementation of the Multi-Sphere T-Matrix method specialised for Complex Amplitude Sensing*, 2026. <https://github.com/MOTEKI-LAB/MSTMforCAS.jl>.

Acknowledgment

This document was prepared with the assistance of Claude (Anthropic). The author assumes full responsibility for the content.